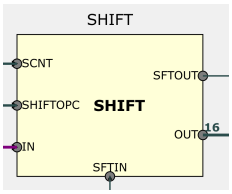
Section 7

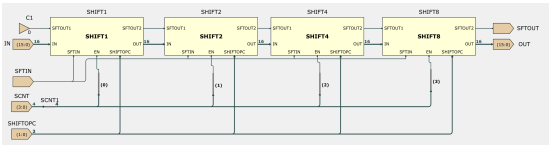
2026年1月29日 星期四 14:38

7 SHIFT instructions

Task 7. Using the LSL, LSR, ASR, and XSR shift instruction definitions from the lectures, predict the waveforms you expect from the code in lab1testshift.txt. Then simulate this program in eep1lab1 and use the waveform viewer to answer the questions below.

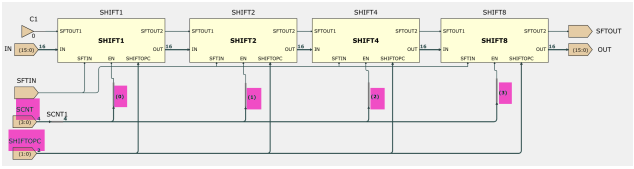
1. Open the shift sheet. Using your tracing skills and the EEP1 Instruction Encoding sheet, deduce the meaning of each signal on the ports of the shift block. You do not need to understand the internals of each individual shifter (SHIFT1, SHIFT2, etc.) in detail; you may assume that each shifter:
- shifts its input by a fixed number of bits when its enable signal EN is 1;
- passes its input through unchanged when EN is 0;
- uses SHIFTOPC to select which of the four shift behaviours (LSL, LSR, ASR, XSR) it performs.





In SHIFT block,
SCNT is amount of bits that the number shifted.
SHIFTOPC controls which SHIFT operation it conducts (from LSL, LSR, ASR, XSR).
SFTIN and SFTOUT control input and output of FLAGCIN.
IN and OUT control input and output of number.

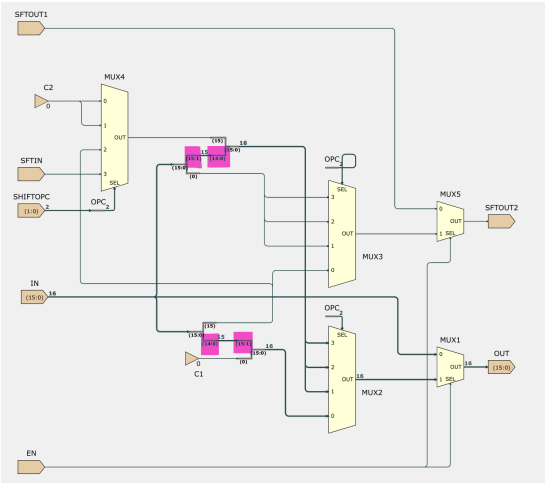
Explain how the combination of SHIFT1, SHIFT2, SHIFT4, and SHIFT8 allows the barrel shifter arrangement to perform the correct shift (type and amount) given the shift count (SCNT) encoded in the instruction.



Each bit of SCNT connects to ENs of SHIFT1;2;4;8 correspondingly.
When EN = 1, shift happens. So total amount of shifts is the same as the amount of SCNT.

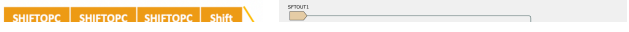
SHIFTOPC is the same into SHIFT1;2;4;8, ensures all SHIFT have the same SHIFT operation.

2. Open the SHIFT1 sheet and explain how its hardware implementation enables it to:
- shift the input data by 1 bit in the required direction.

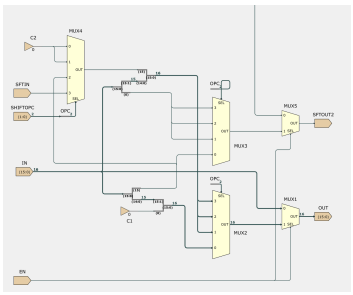


SHIFT1 first leave one bit in IN to make it a 14-bit number, then move all numbers by 1 bit (direction depends on left or right) and add the missing term by whether logic, arithmetic, or extended.

- implement all four shift types (LSL, LSR, ASR, XSR) with appropriate carry-in and carry-out behaviour.



(1:0)	(1)	(0)	
0	0	0	LSL
1	0	1	LSR
2	1	0	ASR
3	1	1	XSR



In MUX2, OPC chooses which type of SHIFT to operate.

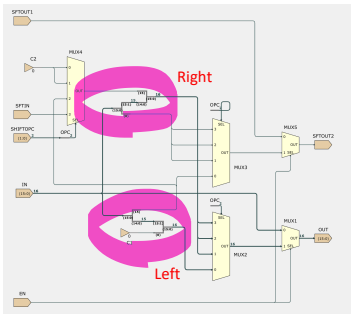
In MUX4, OPC chooses which number (1 or 0) to fill in the missing bit.
LSL & LSR simple logic shift, so = 0;
ASR is arithmetic, so same with 15th bit;
XSR is input from FLAGC.

In MUX3, OPC chooses which carryout to pass to FLAGC.

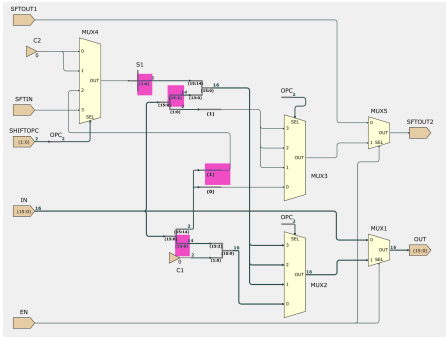
MUX1 & 5 determine whether there is a shift.

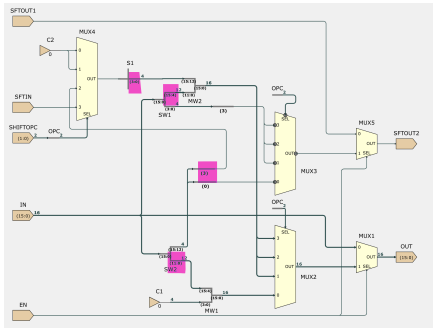
Note: Left shift requires only one shift instruction (Right shift has 3).

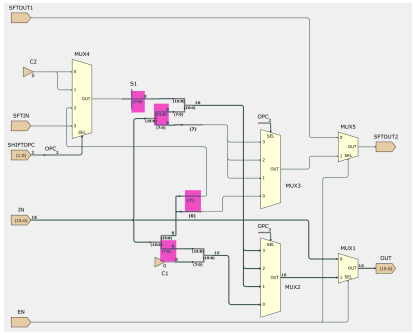
• There are two bus-splitting arrangements, one corresponding to a left shift and one to a right shift. Identify which is which first.



3. Compare the other three shifter sheets (SHIFT2, SHIFT4, SHIFT8) with SHIFT1. Describe how they differ so that they shift by 2, 4, or 8 bits respectively, while otherwise implementing the same four shift types. In particular, explain how the carry-in behaviour of ASR is implemented correctly when the shift amount is greater than 1, and why this is essential for its intended meaning.







These are SHIFT n, they have only a few differences:
1: It leaves n bits and shifts.
2: It copies the adding bit by n times, and then added to the shifted number.

For ASR, the inserted bits are copies of the most significant bit, so the sign of the number is preserved after shifting.

Extension questions
1. Inspect the multiplexer that selects the shifted-in bit for each shift block. What limitation of the XSR instruction does this hardware arrangement reveal?

XSR	multi-word 1 bit right shift (using FlagC to carry bit between words)	Only works if n = 1!

XSR fills the MSB using FLAGCIN, so its result depends on the previous instruction; if FLAGCIN is not explicitly set, the result may be unpredictable.
XSR is fundamentally a single-bit shift operation, and multi-bit shifts must be implemented as repeated SHIFT1, leading to increased execution time compared to other shift operations, and making SHIFT2/4/8 useless.

2. Why is the 1/2/4/8 barrel shifter more efficient than using sixteen SHIFT1 blocks in series? Could you design a barrel shifter that way, and what would be the disadvantages?

Because 1/2/4/8 shifter implements variable shift amounts using only $\log_2(N)$ stages, rather than N stages as would be required by multiple SHIFT1 blocks. Making delay smaller and at the same time, the design easier and smaller.